Design

Nathan Warner

Northern Illinois
University

Computer Science
Northern Illinois University
United States

Contents

Notation

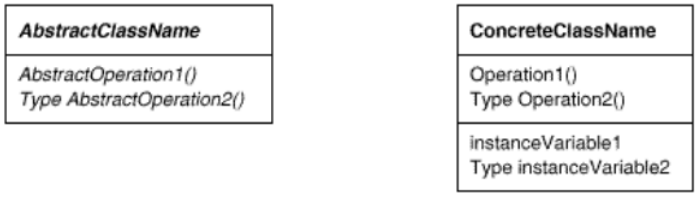
- **Different notations**

  1. **Class diagram**: A class diagram depicts classes, their structure, and the static relationships between them.

     Each design pattern includes at least one class diagram. The other notations are used as needed to supplement the discussion.The class and object diagrams are based on **OMT (Object Modeling Technique)**
  2. **Object diagram**: An object diagram depicts a particular object structure at run-time.

     OMT uses the term ”object diagram” to refer to class diagrams. We use ”object diagram” exclusively to refer to diagrams of object structures
  3. **Interaction diagram**: An interaction diagram shows the flow of requests between objects.

- **Class diagrams**: A class is denoted by a box with the class name in bold type at the top.The key operations of the class appear below the class name. Any instance variables appear below the operations. Type information is optional; we use the C++ convention, which puts the type name before the name of the operation (to signify the return type),instance variable, or actual parameter. Slanted type indicates that the class or operation is abstract.

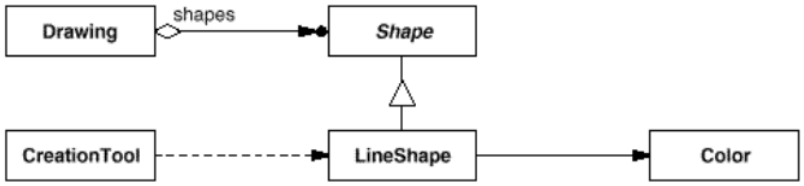


(a) Abstract and concrete classes

In some design patterns it's helpful to see where client classes reference Participant classes. When a pattern includes a Client class as one of its participants (meaning the client has a responsibility in the pattern), the Client appears as an ordinary class. This is true in Flyweight, for example. When the pattern does not include a Client participant (i.e., clients have no responsibilities in the pattern), but including it nevertheless clarifies which pattern participants interact with clients, then the Client class is shown in gray.

(b) Participant Client class (left) and implicit Client class (right)

The OMT notation for class inheritance is a triangle connecting a subclass (LineShape in the figure) to its parent class (Shape). An object reference representing a part-of or aggregation relationship is indicated by an arrow-headed line with a diamond at the base. The arrow points to the class that is aggregated(e.g., Shape). An arrow-headed line without the diamond denotes acquaintance (e.g., a LineShape keeps a reference to a Color object,which other shapes may share). A name for the reference may appear near the base to distinguish it from other references



(c) Class relationships

Another useful thing to show is which classes instantiate which others. We use a dashed arrow-headed line to indicate this, since OMT doesn't support it. We call this the "**creates**" relationship. The arrow points to the class that's instantiated. CreationTool creates LineShape objects.

OMT also defines a filled circle to mean "more than one." When the circle appears at the head of a reference, it means multiple objects are being referenced or aggregated. Figure B.1c shows that Drawing aggregates multiple objects of type Shape.

Finally, we've augmented OMT with pseudocode annotations to let us sketch the implementations of operations.

(d) Pseudocode annotation

- **Object Diagram**: An object diagram shows instances exclusively. It provides a snapshot of the objects in a design pattern. The objects are named "*aSomething*", where Something is the class of the object. Our symbol for an object (modified slightly from standard OMT) is a rounded box with a line separating the object name from any object references. Arrows indicate the object referenced.

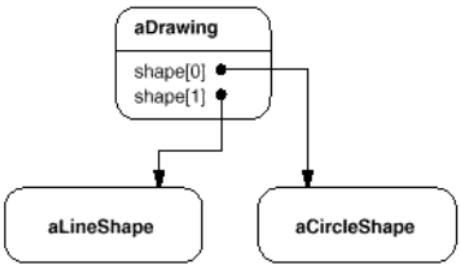
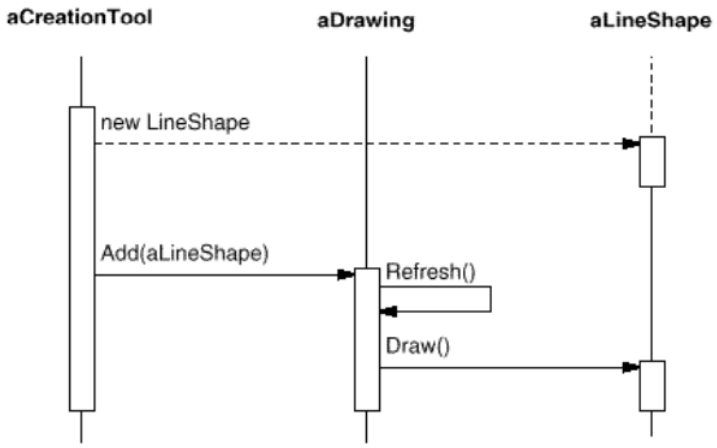


Figure B.2: Object diagram notation

- **Interaction diagrams**: An interaction diagram shows the order in which requests between objects get executed. Below is an interaction diagram that shows how a shape gets added to a drawing.



Time flows from top to bottom in an interaction diagram. A solid vertical line indicates the lifetime of a particular object. The naming convention for objects is the same as for object diagrams; the class name prefixed by the letter "a" (e.g., aShape). If the object doesn't get instantiated until after the beginning of time as recorded in the diagram, then its vertical line appears dashed until the point of creation.

A vertical rectangle shows that an object is active; that is, it is handling a request. The operation can send requests to other objects;these are indicated with a horizontal arrow pointing to the receiving object. The name of the request is shown above the arrow. A request to create an object is shown with a dashed arrow-headed line. A request to the sending object itself points back to the sender.

Introduction

- **Principles of OOP**:
  1. **Encapsulation**: The practice of bundling data (variables) and the methods (functions) that operate on that data into a single unit (a class), while restricting direct access to some of the object's components.
  2. **Abstraction**: The process of hiding complex implementation details and exposing only the essential features or functionalities to the user.
  3. **Inheritance**: A mechanism that allows one class (the child or subclass) to acquire the properties and behaviors of another class (the parent or superclass).
  4. **Polymorphism**: The ability of different objects to respond to the same method call in their own unique way.
- **Parameterized types**: In object-oriented programming (OOP), a parameterized type—more commonly known as a generic or a template—is a class, interface, or method that is defined with placeholder variables in place of actual data types. This architectural feature enables developers to pass specific data types as arguments to a class or method at the time of instantiation or invocation.
- **Object acquaintance**: In Object-Oriented Programming (OOP), object acquaintance (often called an "association" or "using" relationship) is a loose connection where one object merely knows about another to perform a task.
  - **No Ownership**: The "acquainted" object is usually passed as a method parameter or stored in a temporary reference. It does not own the other object and is not responsible for its creation or destruction.
  - **Loosely Coupled**: It allows objects to collaborate and call methods on each other without being strictly tied to their lifecycles.
  - **Implementation**: It is typically implemented via pointers, references, or interfaces.
- **Objects**: In Object-Oriented Programming (OOP), an object is represented as a self-contained unit that combines state (data) and behavior (functions). It is a concrete instance of a blueprint known as a class.
  - **State:** Represented by attributes, fields, or properties.
  - **Behavior:** Represented by methods or functions.
  - **Identity:** A unique address or identifier in memory.
- **Dependent objects**: A dependent object is an object that relies on another object (called its dependency) to function properly or complete its tasks.
- **Independent objects**: An independent object refers to an object that holds its own distinct state and behavior, and can operate or be modified without affecting other objects. It represents a modular, self-contained piece of software that communicates with other objects through messages (methods).
- **Abstract vs concrete classes**:
  - **Abstract class**: An abstract class is an incomplete blueprint that cannot be instantiated directly and often contains undefined methods.
  - **Concrete class**: A concrete class is a complete, fully-implemented class that can be instantiated directly to create objects.
- **Protected constructor**: It is reasonable for an abstract base class to have a protected constructor, but it is not strictly required. Because an abstract class cannot be instantiated directly anyway, a call to the constructor is already an error.

However, a protected constructor is often used to **communicate intent**

- **Lazy initialization**: Lazy initialization is a creational design pattern in object-oriented programming (OOP) that delays the creation of an object, the calculation of a value, or an expensive process until the exact moment it is first needed

  The pattern is typically built by wrapping a private field with an accessor method or property getter.

  1. The field is initially set to null or a placeholder.
  2. When the getter is called, it checks if the instance already exists.
  3. If it exists, it returns it instantly.
  4. If it is null, it initializes the object, saves it to the field, and returns it just-in-time.

  Note: Eager initialization is when the initialization happens at the time of request.

- **Participant / Client**: In software architecture, the participant and client classes represent distinct roles. A client acts as the interface initiating an interaction (e.g., an AWS or API connection), while a participant represents an active entity within that session (e.g., a person, agent, or device in a chat or RTC room).

  - **Client**: The client class establishes a connection and handles the communication protocol, authentication, and requests between your application and a backend service.
  - **Participant**: The participant class represents a specific user, agent, or automated bot actively involved in the established session or context.

- **What is a design pattern**: Each pattern describes a problem which occurs over and over again in our environment, and then describes the core of the solution to that problem, in such a way that you can use this solution a million times over, without ever doing it the same way twice

- **Four essential elements**:

  1. **Pattern name**: The pattern name is a handle we can use to describe a design problem, its solutions, and consequences in a word or two. Naming a pattern immediately increases our design vocabulary. It lets us design at a higher level of abstraction. Having a vocabulary for patterns lets us talk about them with our colleagues, in our documentation, and even to ourselves. It makes it easier to think about designs and to communicate them and their trade-offs to others. Finding good names has been one of the hardest parts of developing our catalog.
  2. **Problem**: The problem describes when to apply the pattern.It explains the problem and its context. It might describe specific design problems such as how to represent algorithms as objects.It might describe class or object structures that are symptomatic of an inflexible design. Sometimes the problem will include a list of conditions that must be met before it makes sense to apply the pattern.
  3. **Solution**: The solution describes the elements that make up the design, their relationships, responsibilities, and collaborations. The solution doesn't describe a particular concrete design or implementation, because a pattern is like a template that can be applied in many different situations. Instead, the pattern provides an abstract description of a design problem and how a general arrangement of elements (classes and objects in our case) solves it.

4. **Consequences**: The consequences are the results and trade-offs of applying the pattern.Though consequences are often unvoiced when we describe design decisions, they are critical for evaluating design alternatives and for understanding the costs and benefits of applying the pattern. The consequences for software often concern space and time trade-offs. They may address language and implementation issues as well. Since reuse is often a factor in object-oriented design, the consequences of a pattern include its impact on a system's flexibility, extensibility, or portability. Listing these consequences explicitly helps you understand and evaluate them.

- **MVC**: MVC consists of three kinds of objects. The Model is the application object, the View is its screen presentation, and the Controller defines the way the user interface reacts to user input.

  MVC decouples views and models by establishing a subscribe/notify protocol between them. A view must ensure that its appearance reflects the state of the model. Whenever the model's data changes, the model notifies views that depend on it. In response, each view gets an opportunity to update itself. This approach lets you attach multiple views to a model to provide different presentations. You can also create new views for a model without rewriting it.

  This design is applicable to a more general problem: decoupling objects so that changes to one can affect any number of others without requiring the changed object to know details of the others.

  MVC also lets you change the way a view responds to user input without changing its visual presentation. You might want to change the way it responds to the keyboard, for example, or have it use a pop-up menu instead of command keys. MVC encapsulates the response mechanism in a Controller object. There is a class hierarchy of controllers, making it easy to create a new controller as a variation on an existing one.

  A view uses an instance of a Controller subclass to implement a particular response strategy; to implement a different strategy, simply replace the instance with a different kind of controller. It's even possible to change a view's controller at run-time to let the view change the way it responds to user input. For example, a view can be disabled so that it doesn't accept input simply by giving it a controller that ignores input events.

  The main relationships in MVC are given by the Observer, Composite, and Strategy design patterns. MVC uses other design patterns, such as Factory Method (121) to specify the default controller class for a view and Decorator (196) to add scrolling to a view.

- **Describing design patterns**: We describe design patterns using a consistent format. Each pattern is divided into sections according to the following template. The template lends a uniform structure to the information, making design patterns easier to learn, compare, and use.

  1. **Pattern name and classification**: The pattern's name conveys the essence of the pattern succinctly. A good name is vital, because it will become part of your design vocabulary.
  2. **Intent**: A short statement that answers the following questions: What does the design pattern do? What is its rationale and intent? What particular design issue or problem does it address?

3. **Also known as**: Other well-known names for the pattern, if any.
4. **Motivation**: A scenario that illustrates a design problem and how the class and object structures in the pattern solve the problem. The scenario will help you understand the more abstract description of the pattern that follows.
5. **Applicability**: What are the situations in which the design pattern can be applied? What are examples of poor designs that the pattern can address? How can you recognize these situations?
6. **Structure**: A graphical representation of the classes in the pattern using a notation based on the **Object Modeling Technique (OMT)**. We also use **interaction diagrams** to illustrate sequences of requests and collaborations between objects.
7. **Participants**: The classes and/or objects participating in the design pattern and their responsibilities.
8. **Collaborations**: How the participants collaborate to carry out their responsibilities.
9. **Consequences**: How does the pattern support its objectives? What are the trade-offs and results of using the pattern? What aspect of system structure does it let you vary independently?
10. **Implementation**: What pitfalls, hints, or techniques should you be aware of when implementing the pattern? Are there language-specific issues?
11. **Sample code**: Code fragments that illustrate how you might implement the pattern in C++ or Smalltalk.
12. **Known uses**: Examples of the pattern found in real systems.
13. **Related patterns**: What design patterns are closely related to this one? What are the important differences? With which other patterns should this one be used?

- **The catalog of design patterns**:
  - **Abstract Factory**: Provide an interface for creating families of related or dependent objects without specifying their concrete classes.
  - **Adapter**: Convert the interface of a class into another interface clients expect. Adapter lets classes work together that couldn't otherwise because of incompatible interfaces.
  - **Bridge**: Decouple an abstraction from its implementation so that the two can vary independently.
  - **Builder**: Separate the construction of a complex object from its representation so that the same construction process can create different representations.
  - **Chain of Responsibility**: Avoid coupling the sender of a request to its receiver by giving more than one object a chance to handle the request. Chain the receiving objects and pass the request along the chain until an object handles it.
  - **Command**: Encapsulate a request as an object, thereby letting you parameterize clients with different requests, queue or log requests, and support undoable operations.
  - **Composite**: Compose objects into tree structures to represent part-whole hierarchies. Composite lets clients treat individual objects and compositions of objects uniformly.
  - **Decorator**: Attach additional responsibilities to an object dynamically. Decorators provide a flexible alternative to subclassing for extending functionality.

- **Facade**: Provide a unified interface to a set of interfaces in a subsystem. Facade defines a higher-level interface that makes the subsystem easier to use.
- **Factory Method**: Define an interface for creating an object, but let subclasses decide which class to instantiate. Factory Method lets a class defer instantiation to subclasses.
- **Flyweight**: Use sharing to support large numbers of fine-grained objects efficiently.
- **Interpreter**: Given a language, define a representation for its grammar along with an interpreter that uses the representation to interpret sentences in the language.
- **Iterator**: Provide a way to access the elements of an aggregate object sequentially without exposing its underlying representation.
- **Mediator**: Define an object that encapsulates how a set of objects interact. Mediator promotes loose coupling by keeping objects from referring to each other explicitly, and it lets you vary their interaction independently.
- **Memento**: Without violating encapsulation, capture and externalize an object's internal state so that the object can be restored to this state later.
- **Observer**: Define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.
- **Prototype**: Specify the kinds of objects to create using a prototypical instance, and create new objects by copying this prototype.
- **Proxy**: Provide a surrogate or placeholder for another object to control access to it.
- **Singleton**: Ensure a class only has one instance, and provide a global point of access to it.
- **State**: Allow an object to alter its behavior when its internal state changes. The object will appear to change its class.
- **Strategy**: Define a family of algorithms, encapsulate each one, and make them interchangeable. Strategy lets the algorithm vary independently from clients that use it.
- **Template Method**: Define the skeleton of an algorithm in an operation, deferring some steps to subclasses. Template Method lets subclasses redefine certain steps of an algorithm without changing the algorithm's structure.
- **Visitor**: Represent an operation to be performed on the elements of an object structure. Visitor lets you define a new operation without changing the classes of the elements on which it operates.

- **Organize the catalog**: Design patterns vary in their granularity and level of abstraction. Because there are many design patterns, we need a way to organize them. This section classifies design patterns so that we can refer to families of related patterns. The classification helps you learn the patterns in the catalog faster, and it can direct efforts to find new patterns as well.

  We classify design patterns by two criteria

  1. **Purpose**: The **first criterion**, called purpose, reflects what a pattern does. Patterns can have either **creational**, **structural**, or **behavioral** purpose. Creational patterns concern the process of object creation. Structural patterns deal with the composition of classes or objects. Behavioral patterns characterize the ways in which classes or objects interact and distribute responsibility.

2. **Scope**: The second criterion, called **scope**, specifies whether the pattern applies primarily to classes or to objects. Class patterns deal with relationships between classes and their subclasses. These relationships are established through inheritance, so they are static—fixed at compile-time. Object patterns deal with object relationships, which can be changed at run-time and are more dynamic. Almost all patterns use inheritance to some extent. So the only patterns labeled "class patterns" are those that focus on class relationships. Note that most patterns are in the Object scope.

		Purpose		
		Creational	**Structural**	**Behavioral**
Scope	**Class**	Factory Method (107)	Adapter (class) (139)	Interpreter (243) Template Method (325)
	Object	Abstract Factory (87) Builder (97) Prototype (117) Singleton (127)	Adapter (object) (139) Bridge (151) Composite (163) Decorator (175) Facade (185) Flyweight (195) Proxy (207)	Chain of Responsibility (223) Command (233) Iterator (257) Mediator (273) Memento (283) Observer (293) State (305) Strategy (315) Visitor (331)

Table 1.1: Design pattern space

Creational class patterns defer some part of object creation to subclasses, while Creational object patterns defer it to another object.

The Structural class patterns use inheritance to compose classes, while the Structural object patterns describe ways to assemble objects.

The Behavioral class patterns use inheritance to describe algorithms and flow of control, whereas the Behavioral object patterns describe how a group of objects cooperate to perform a task that no single object can carry out alone.

There are other ways to organize the patterns. Some patterns are often used together. For example, Composite is often used with Iterator or Visitor. Some patterns are alternatives: Prototype is often an alternative to Abstract Factory. Some patterns result in similar designs even though the patterns have different intents. For example, the structure diagrams of Composite and Decorator are similar.

Yet another way to organize design patterns is according to how they reference each other in their "Related Patterns" sections.

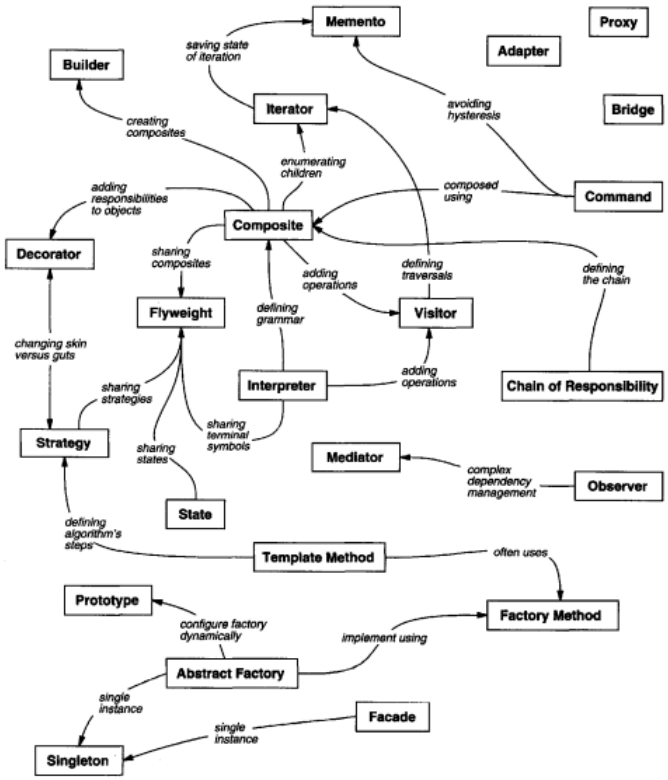


Figure 1.1: Design pattern relationships

- **Finding Appropriate Objects**: Object-oriented programs are made up of objects. An object packages both data and the procedures that operate on that data. The procedures are typically called **methods** or operations. An object performs an operation when it receives a request (or **message**) from a **client**.

  Requests are the only way to get an object to execute an operation. Operations are the only way to change an object's internal data. Because of these restrictions, the object's internal state is said to be encapsulated; it cannot be accessed directly, and its representation is invisible from outside the object.

  The hard part about object-oriented design is decomposing a system into objects. The task is difficult because many factors come into play: encapsulation, granularity, dependency, flexibility, performance, evolution, reusability, and on and on. They all influence the decomposition, often in conflicting ways.

  Object-oriented design methodologies favor many different approaches. You can;

  (a) Write a problem statement, single out the nouns and verbs, and create corresponding classes and operations.

  (b) Focus on the collaborations and responsibilities in your system.

  (c) Model the real world and translate the objects found during analysis into design. There will always be disagreement on which approach is best.

Many objects in a design come from the analysis model. But object-oriented designs often end up with classes that have no counterparts in the real world. Strict modeling of the real world leads to a system that reflects today's realities but not necessarily tomorrow's. The abstractions that emerge during design are key to making a design flexible.

Design patterns help you identify less-obvious abstractions and the objects that can capture them.

- **Determining Object Granularity**: Objects can vary tremendously in size and number. They can represent everything down to the hardware or all the way up to entire applications. How do we decide what should be an object?
- **Specifying Object Interfaces:** Every operation declared by an object specifies the operation's name, the objects it takes as parameters, and the operation's return value. This is known as the operation's signature. The set of all signatures defined by an object's operations is called the interface to the object. An object's interface characterizes the complete set of requests that can be sent to the object. Any request that matches a signature in the object's interface may be sent to the object.

  A type is a name used to denote a particular interface. We speak of an object as having the type "Window" if it accepts all requests for the operations defined in the interface named "Window." An object may have many types, and widely different objects can share a type. Part of an object's interface may be characterized by one type, and other parts by other types. Two objects of the same type need only share parts of their interfaces. Interfaces can contain other interfaces as subsets. We say that a type is a subtype of another if its interface contains the interface of its supertype. Often we speak of a subtype inheriting the interface of its supertype.

  Interfaces are fundamental in object-oriented systems. Objects are known only through their interfaces. There is no way to know anything about an object or to ask it to do anything without going through its interface. An object's interface says nothing about its implementation, different objects are free to implement requests differently. That means two objects having completely different implementations can have identical interfaces.

  When a request is sent to an object, the particular operation that's performed depends on both the request and the receiving object. Different objects that support identical requests may have different implementations of the operations that fulfill these requests. The run-time association of a request to an object and one of its operations is known as dynamic binding.

  Dynamic binding means that issuing a request doesn't commit you to a particular implementation until run-time.

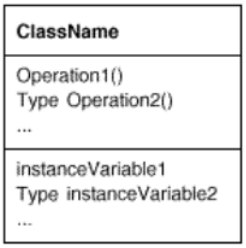
  Moreover, dynamic binding lets you substitute objects that have identical interfaces for each other at run-time. This substitutability is known as polymorphism, and it's a key concept in object-oriented systems. It lets a client object make few assumptions about other objects beyond supporting a particular interface. Polymorphism simplifies the definitions of clients, decouples objects from each other, and lets them vary their relationships to each other at run-time.

  Design patterns help you define interfaces by identifying their key elements and the kinds of data that get sent across an interface. A design pattern might also tell you what not to put in the interface.

Design patterns also specify relationships between interfaces. In particular, they often require some classes to have similar interfaces, or they place constraints on the interfaces of some classes.

- **Specifying Object Implementations**: So far we've said little about how we actually define an object. An object's implementation is defined by its class. The class specifies the object's internal data and representation and defines the operations the object can perform.

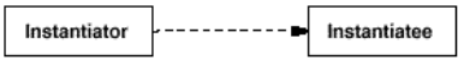
  Our OMT-based notation depicts a class as a rectangle with the class name in bold. Operations appear in normal type below the class name. Any data that the class defines comes after the operations. Lines separate the class name from the operations and the operations from the data.



Return types and instance variable types are optional, since we don't assume a statically typed implementation language.

Objects are created by **instantiating** a class. The object is said to be an **instance** of the class. The process of instantiating a class allocates storage for the object's internal data (made up of instance variables) and associates the operations with these data. Many similar instances of an object can be created by instantiating a class.

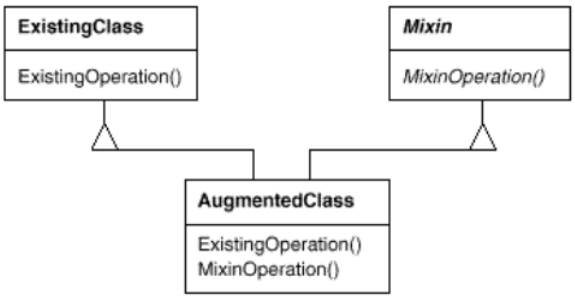
A dashed arrowhead line indicates a class that instantiates objects of another class. The arrow points to the class of the instantiated objects



An abstract class is one whose main purpose is to define a common interface for its subclasses. An abstract class will defer some or all of its implementation to operations defined in subclasses; hence an abstract class cannot be instantiated.

The operations that an abstract class declares but doesn't implement are called abstract operations. Classes that aren't abstract are called concrete classes.

A mixin class is a class that's intended to provide an optional interface or functionality to other classes. It's similar to an abstract class in that it's not intended to be instantiated. Mixin classes require multiple inheritance:



- **Class versus interface inheritance**: It's important to understand the difference between an object's class and its type. An object's class defines how the object is implemented. The class defines the object's internal state and the implementation of its operations. In contrast, an object's type only refers to its interface—the set of requests to which it can respond. An object can have many types, and objects of different classes can have the same type.

  Of course, there's a close relationship between class and type. Because a class defines the operations an object can perform, it also defines the object's type. When we say that an object is an instance of a class, we imply that the object supports the interface defined by the class.

  It's also important to understand the difference between class inheritance and interface inheritance (or subtyping). Class inheritance defines an object's implementation in terms of another object's implementation. In short, it's a mechanism for code and representation sharing. In contrast, interface inheritance (or subtyping) describes when an object can be used in place of another.

  It's easy to confuse these two concepts, because many languages don't make the distinction explicit. In languages like C++ and Eiffel, inheritance means both interface and implementation inheritance. The standard way to inherit an interface in C++ is to inherit publicly from a class that has (pure) virtual member functions. Pure interface inheritance can be approximated in C++ by inheriting publicly from pure abstract classes. Pure implementation or class inheritance can be approximated with private inheritance.

  Although most programming languages don't support the distinction between interface and implementation inheritance, people make the distinction in practice.

- **Programming to an Interface, not an Implementation**: Class inheritance is basically just a mechanism for extending an application's functionality by reusing functionality in parent classes. It lets you define a new kind of object rapidly in terms of an old one. It lets you get new implementations almost for free, inheriting most of what you need from existing classes.

  However, implementation reuse is only half the story. Inheritance's ability to define families of objects with identical interfaces (usually by inheriting from an abstract class) is also important. Why? Because polymorphism depends on it.

When inheritance is used carefully (some will say properly), all classes derived from an abstract class will share its interface. This implies that a subclass merely adds or overrides operations and does not hide operations of the parent class. All subclasses can then respond to the requests in the interface of this abstract class, making them all subtypes of the abstract class.

There are two benefits to manipulating objects solely in terms of the interface defined by abstract classes:

1. Clients remain unaware of the specific types of objects they use, as long as the objects adhere to the interface that clients expect
2. Clients remain unaware of the classes that implement these objects. Clients only know about the abstract class(es) defining the interface.

This so greatly reduces implementation dependencies between subsystems that it leads to the following principle of reusable object-oriented design:

"*program to an interface, not an implementation*"

Don't declare variables to be instances of particular concrete classes. Instead, commit only to an interface defined by an abstract class. You will find this to be a common theme of the design patterns in this book.

You have to instantiate concrete classes (that is, specify a particular implementation) somewhere in your system, of course, and the creational patterns (Abstract Factory (99), Builder (110), Factory Method (121), Prototype (133), and Singleton (144) let you do just that. By abstracting the process of object creation, these patterns give you different ways to associate an interface with its implementation transparently at instantiation. Creational patterns ensure that your system is written in terms of interfaces, not implementations.

- **Inheritance versus Composition**: The two most common techniques for reusing functionality in object-oriented systems are class inheritance and object composition. As we've explained, class inheritance lets you define the implementation of one class in terms of another's. Reuse by subclassing is often referred to as white-box reuse. The term "white-box" refers to visibility: With inheritance, the internals of parent classes are often visible to subclasses.

  Object composition is an alternative to class inheritance. Here, new functionality is obtained by assembling or composing objects to get more complex functionality. Object composition requires that the objects being composed have well-defined interfaces. This style of reuse is called black-box reuse, because no internal details of objects are visible. Objects appear only as "black boxes."

  Inheritance and composition each have their advantages and disadvantages. Class inheritance is defined statically at compile-time and is straightforward to use, since it's supported directly by the programming language. Class inheritance also makes it easier to modify the implementation being reused. When a subclass overrides some but not all operations, it can affect the operations it inherits as well, assuming they call the overridden operations

But class inheritance has some disadvantages, too. First, you can't change the implementations inherited from parent classes at run-time, because inheritance is defined at compile-time. Second, and generally worse, parent classes often define at least part of their subclasses' physical representation. Because inheritance exposes a subclass to details of its parent's implementation, it's often said that "inheritance breaks encapsulation". he implementation of a subclass becomes so bound up with the implementation of its parent class that any change in the parent's implementation will force the subclass to change.

Implementation dependencies can cause problems when you're trying to reuse a subclass. Should any aspect of the inherited implementation not be appropriate for new problem domains, the parent class must be rewritten or replaced by something more appropriate. This dependency limits flexibility and ultimately reusability. One cure for this is to inherit only from abstract classes, since they usually provide little or no implementation.

Object composition is defined dynamically at run-time through objects acquiring references to other objects. Composition requires objects to respect each others' interfaces, which in turn requires carefully designed interfaces that don't stop you from using one object with many others. But there is a payoff. Because objects are accessed solely through their interfaces, we don't break encapsulation. Any object can be replaced at run-time by another as long as it has the same type. Moreover, because an object's implementation will be written in terms of object interfaces, there are substantially fewer implementation dependencies.

Object composition has another effect on system design. Favoring object composition over class inheritance helps you keep each class encapsulated and focused on one task. Your classes and class hierarchies will remain small and will be less likely to grow into unmanageable monsters. On the other hand, a design based on object composition will have more objects (if fewer classes), and the system's behavior will depend on their interrelationships instead of being defined in one class.

That leads us to our second principle of object-oriented design:

"*Favor object composition over class inheritance*"

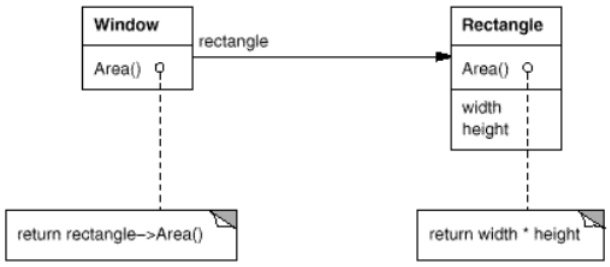
Ideally, you shouldn't have to create new components to achieve reuse. You should be able to get all the functionality you need just by assembling existing components through object composition. But this is rarely the case, because the set of available components is never quite rich enough in practice. Reuse by inheritance makes it easier to make new components that can be composed with old ones. Inheritance and object composition thus work together.

- **Delegation**: Delegation is a way of making composition as powerful for reuse as inheritance. In delegation, two objects are involved in handling a request: a receiving object delegates operations to its **delegate.**

This is analogous to subclasses deferring requests to parent classes. But with inheritance, an inherited operation can always refer to the receiving object through the this member variable in C++ and self in Smalltalk. To achieve the same effect with delegation, the receiver passes itself to the delegate to let the delegated operation refer to the receiver.

For example, instead of making class Window a subclass of Rectangle (because windows happen to be rectangular), the Window class might reuse the behavior of Rectangle by keeping a Rectangle instance variable and delegating Rectangle-specific behavior to it. In other words, instead of a Window being a Rectangle, it would have a Rectangle. Window must now forward requests to its Rectangle instance explicitly, whereas before it would have inherited those operations.

The following diagram depicts the Window class delegating its Area operation to a Rectangle instance.



The main advantage of delegation is that it makes it easy to compose behaviors at run-time and to change the way they're composed. Our window can become circular at run-time simply by replacing its Rectangle instance with a Circle instance, assuming Rectangle and Circle have the same type.

Delegation has a disadvantage it shares with other techniques that make software more flexible through object composition: Dynamic, highly parameterized software is harder to understand than more static software. There are also run-time inefficiencies, but the human inefficiencies are more important in the long run. Delegation is a good design choice only when it simplifies more than it complicates. It isn't easy to give rules that tell you exactly when to use delegation, because how effective it will be depends on the context and on how much experience you have with it. Delegation works best when it's used in highly stylized ways—that is, in standard patterns

- **Inheritance versus Parameterized Types**: Another (not strictly object-oriented) technique for reusing functionality is through parameterized types, also known as generics (Ada, Eiffel) and templates (C++). This technique lets you define a type without specifying all the other types it uses. The unspecified types are supplied as parameters at the point of use. For example, a List class can be parameterized by the type of elements it contains. To declare a list of integers, you supply the type "integer" as a parameter to the List parameterized type. To declare a list of String objects, you supply the "String" type as a parameter. The language implementation will create a customized version of the List class template for each type of element.

Parameterized types give us a third way (in addition to class inheritance and object composition) to compose behavior in object-oriented systems. Many designs can be implemented using any of these three techniques. To parameterize a sorting routine by the operation it uses to compare elements, we could make the comparison one of:

1. an operation implemented by subclasses (an application of Template Method (360),

2. the responsibility of an object that's passed to the sorting routine (Strategy (349), or
3. an argument of a C++ template or Ada generic that specifies the name of the function to call to compare the elements.

There are important differences between these techniques. Object composition lets you change the behavior being composed at run-time, but it also requires indirection and can be less efficient. Inheritance lets you provide default implementations for operations and lets subclasses override them. Parameterized types let you change the types that a class can use. But neither inheritance nor parameterized types can change at run-time. Which approach is best depends on your design and implementation constraints.

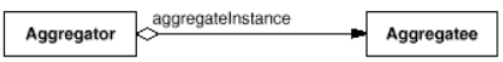
None of the patterns we see in this document concerns parameterized types, though we use them on occasion to customize a pattern's C++ implementation. Parameterized types aren't needed at all in a language like Smalltalk that doesn't have compile-time type checking

- **Relating Run-Time and Compile-Time Structures**: An object-oriented program's run-time structure often bears little resemblance to its code structure. The code structure is frozen at compile-time; it consists of classes in fixed inheritance relationships. A program's run-time structure consists of rapidly changing networks of communicating objects. In fact, the two structures are largely independent.

  Consider the distinction between object aggregation and acquaintance and how differently they manifest themselves at compile- and run-times. Aggregation implies that one object owns or is responsible for another object. Generally we speak of an object having or being part of another object. Aggregation implies that an aggregate object and its owner have identical lifetimes.

  Acquaintance implies that an object merely knows of another object. Sometimes acquaintance is called "association" or the "using" relationship. Acquainted objects may request operations of each other, but they aren't responsible for each other. Acquaintance is a weaker relationship than aggregation and suggests much looser coupling between objects.

  In our diagrams, a plain arrowhead line denotes acquaintance. An arrowhead line with a diamond at its base denotes aggregation:



  It's easy to confuse aggregation and acquaintance, because they are often implemented in the same way. In Smalltalk, all variables are references to other objects. There's no distinction in the programming language between aggregation and acquaintance. In C++, aggregation can be implemented by defining member variables that are real instances, but it's more common to define them as pointers or references to instances. Acquaintance is implemented with pointers and references as well.

Ultimately, acquaintance and aggregation are determined more by intent than by explicit language mechanisms. The distinction may be hard to see in the compile-time structure, but it's significant. Aggregation relationships tend to be fewer and more permanent than acquaintance. Acquaintances, in contrast, are made and remade more frequently, sometimes existing only for the duration of an operation. Acquaintances are more dynamic as well, making them more difficult to discern in the source code

With such disparity between a program's run-time and compile-time structures, it's clear that code won't reveal everything about how a system will work. The system's run-time structure must be imposed more by the designer than the language. The relationships between objects and their types must be designed with great care, because they determine how good or bad the run-time structure is.

Many design patterns (in particular those that have object scope) capture the distinction between compile-time and run-time structures explicitly. Composite (183) and Decorator (196) are especially useful for building complex run-time structures. Observer (326) involves run-time structures that are often hard to understand unless you know the pattern. Chain of Responsibility (251) also results in communication patterns that inheritance doesn't reveal. In general, the run-time structures aren't clear from the code until you understand the patterns.

- **Designing for change**: The key to maximizing reuse lies in anticipating new requirements and changes to existing requirements, and in designing your systems so that they can evolve accordingly.

To design the system so that it's robust to such changes, you must consider how the system might need to change over its lifetime. A design that doesn't take change into account risks major redesign in the future. Those changes might involve class redefinition and reimplementation, client modification, and retesting.

Redesign affects many parts of the software system, and unanticipated changes are invariably expensive

Design patterns help you avoid this by ensuring that a system can change in specific ways. Each design pattern lets some aspect of system structure vary independently of other aspects, thereby making a system more robust to a particular kind of change.

Here are some common causes of redesign along with the design pattern(s) that address them:

1. Creating an object by specifying a class explicitly. Specifying a class name when you create an object commits you to a particular implementation instead of a particular interface. This commitment can complicate future changes. To avoid it, create objects indirectly.
   - **Design patterns**: Abstract Factory (99), Factory Method (121), Prototype (133).
2. Dependence on specific operations. When you specify a particular operation, you commit to one way of satisfying a request. By avoiding hard-coded requests, you make it easier to change the way a request gets satisfied both at compile-time and at run-time.
   - **Design patterns**: Chain of Responsibility (251), Command (263).

3. Dependence on hardware and software platform. External operating system interfaces and application programming interfaces (APIs) are different on different hardware and software platforms. Software that depends on a particular platform will be harder to port to other platforms. It may even be difficult to keep it up to date on its native platform. It's important therefore to design your system to limit its platform dependencies.
   - **Design patterns**: Abstract Factory (99), Bridge (171).
4. Dependence on object representations or implementations. Clients that know how an object is represented, stored, located, or implemented might need to be changed when the object changes. Hiding this information from clients keeps changes from cascading.
   - **Design patterns**: Abstract Factory (99), Bridge (171), Memento (316), Proxy (233).
5. Algorithmic dependencies. Algorithms are often extended, optimized, and replaced during development and reuse. Objects that depend on an algorithm will have to change when the algorithm changes. Therefore algorithms that are likely to change should be isolated.
   - **Design patterns**: Builder (110), Iterator (289), Strategy (349), Template Method (360), Visitor (366).
6. Tight coupling. Classes that are tightly coupled are hard to reuse in isolation, since they depend on each other. Tight coupling leads to monolithic systems, where you can't change or remove a class without understanding and changing many other classes. The system becomes a dense mass that's hard to learn, port, and maintain. Loose coupling increases the probability that a class can be reused by itself and that a system can be learned, ported, modified, and extended more easily. Design patterns use techniques such as abstract coupling and layering to promote loosely coupled systems.
   - **Design patterns**: Abstract Factory (99), Bridge (171), Chain of Responsibility (251), Command (263), Facade (208), Mediator (305), Observer (326).
7. Extending functionality by subclassing. Customizing an object by subclassing often isn't easy. Every new class has a fixed implementation overhead (initialization, finalization, etc.). Defining a subclass also requires an in-depth understanding of the parent class. For example, overriding one operation might require overriding another. An overridden operation might be required to call an inherited operation. And subclassing can lead to an explosion of classes, because you might have to introduce many new subclasses for even a simple extension. Object composition in general and delegation in particular provide flexible alternatives to inheritance for combining behavior. New functionality can be added to an application by composing existing objects in new ways rather than by defining new subclasses of existing classes. On the other hand, heavy use of object composition can make designs harder to understand. Many design patterns produce designs in which you can introduce customized functionality just by defining one subclass and composing its instances with existing ones.
   - **Design patterns**: Bridge (171), Chain of Responsibility (251), Composite (183), Decorator (196), Observer (326), Strategy (349).
8. Inability to alter classes conveniently. Sometimes you have to modify a class that can't be modified conveniently. Perhaps you need the source code and don't have it (as may be the case with a commercial class library). Or maybe any change would require modifying lots of existing subclasses. Design patterns offer ways to modify classes in such circumstances.
   - **Design patterns**: Adapter (157), Decorator (196), Visitor (366).

- **Application Programs**: If you're building an application program such as a document editor or spreadsheet, then internal reuse, maintainability, and extension are high priorities. Internal reuse ensures that you don't design and implement any more than you have to. Design patterns that reduce dependencies can increase internal reuse. Looser coupling boosts the likelihood that one class of object can cooperate with several others. For example, when you eliminate dependencies on specific operations by isolating and encapsulating each operation, you make it easier to reuse an operation in different contexts. The same thing can happen when you remove algorithmic and representational dependencies too.

  Design patterns also make an application more maintainable when they're used to limit platform dependencies and to layer a system. They enhance extensibility by showing you how to extend class hierarchies and how to exploit object composition. Reduced coupling also enhances extensibility. Extending a class in isolation is easier if the class doesn't depend on lots of other classes

- **Toolkits**: Often an application will incorporate classes from one or more libraries of predefined classes called toolkits. A toolkit is a set of related and reusable classes designed to provide useful, general-purpose functionality. An example of a toolkit is a set of collection classes for lists, associative tables, stacks, and the like. The C++ I/O stream library is another example. Toolkits don't impose a particular design on your application; they just provide functionality that can help your application do its job. They let you as an implementer avoid recoding common functionality. Toolkits emphasize code reuse. They are the object-oriented equivalent of subroutine libraries.

  Toolkit design is arguably harder than application design, because toolkits have to work in many applications to be useful. Moreover, the toolkit writer isn't in a position to know what those applications will be or their special needs.

  That makes it all the more important to avoid assumptions and dependencies that can limit the toolkit's flexibility and consequently its applicability and effectiveness

- **Frameworks**: A framework is a set of cooperating classes that make up a reusable design for a specific class of software. For example, a framework can be geared toward building graphical editors for different domains like artistic drawing, music composition, and mechanical CAD. Another framework can help you build compilers for different programming languages and target machines. Yet another might help you build financial modeling applications. You customize a framework to a particular application by creating application-specific subclasses of abstract classes from the framework.

  The framework dictates the architecture of your application. It will define the overall structure, its partitioning into classes and objects, the key responsibilities thereof, how the classes and objects collaborate, and the thread of control. A framework predefines these design parameters so that you, the application designer/implementer, can concentrate on the specifics of your application. The framework captures the design decisions that are common to its application domain. Frameworks thus emphasize design reuse over code reuse, though a framework will usually include concrete subclasses you can put to work immediately.

  Reuse on this level leads to an inversion of control between the application and the software on which it's based. When you use a toolkit (or a conventional subroutine library for that matter), you write the main body of the application and call the code you want to reuse. When you use a framework, you reuse the main body and write the code it calls. You'll have to write operations with particular names and calling conventions, but that reduces the design decisions you have to make

Not only can you build applications faster as a result, but the applications have similar structures. They are easier to maintain, and they seem more consistent to their users. On the other hand, you lose some creative freedom, since many design decisions have been made for you.

If applications are hard to design, and toolkits are harder, then frameworks are hardest of all. A framework designer gambles that one architecture will work for all applications in the domain. Any substantive change to the framework's design would reduce its benefits considerably, since the framework's main contribution to an application is the architecture it defines. Therefore it's imperative to design the framework to be as flexible and extensible as possible.

Furthermore, because applications are so dependent on the framework for their design, they are particularly sensitive to changes in framework interfaces. As a framework evolves, applications have to evolve with it. That makes loose coupling all the more important; otherwise even a minor change to the framework will have major repercussions.

The design issues just discussed are most critical to framework design. A framework that addresses them using design patterns is far more likely to achieve high levels of design and code reuse than one that doesn't. Mature frameworks usually incorporate several design patterns. The patterns help make the framework's architecture suitable to many different applications without redesign

An added benefit comes when the framework is documented with the design patterns it uses. People who know the patterns gain insight into the framework faster. Even people who don't know the patterns can benefit from the structure they lend to the framework's documentation. Enhancing documentation is important for all types of software, but it's particularly important for frameworks. Frameworks often pose a steep learning curve that must be overcome before they're useful. While design patterns might not flatten the learning curve entirely, they can make it less steep by making key elements of the framework's design more explicit.

Because patterns and frameworks have some similarities, people often wonder how or even if they differ. They are different in three major ways:

1. Design patterns are more abstract than frameworks. Frameworks can be embodied in code, but only examples of patterns can be embodied in code. A strength of frameworks is that they can be written down in programming languages and not only studied but executed and reused directly. In contrast, the design patterns in this book have to be implemented each time they're used. Design patterns also explain the intent, trade-offs, and consequences of a design.
2. Design patterns are smaller architectural elements than frameworks. A typical framework contains several design patterns, but the reverse is never true.
3. Design patterns are less specialized than frameworks. Frameworks always have a particular application domain. A graphical editor framework might be used in a factory simulation, but it won't be mistaken for a simulation framework. In contrast, the design patterns in this catalog can be used in nearly any kind of application. While more specialized design patterns than ours are certainly possible (say, design patterns for distributed systems or concurrent programming), even these wouldn't dictate an application architecture like a framework would.

- **How to select a design pattern**:

1. Consider how design patterns solve design problems. We previously discussed how design patterns help you find appropriate objects, determine object granularity, specify object interfaces, and several other ways in which design patterns solve design problems. Referring to these discussions can help guide your search for the right pattern.
2. Scan Intent sections. Read through each pattern's intent to find one or more that sound relevant to your problem.
3. Study how patterns interrelate.
4. Study patterns of like purpose.
5. Examine a cause of redesign.
6. Consider what should be variable in your design. This approach is the opposite of focusing on the causes of redesign. Instead of considering what might force a change to a design, consider what you want to be able to change without redesign. The focus here is on encapsulating the concept that varies, a theme of many design patterns.

- **What can vary**:

Purpose	Design Pattern	Aspect(s) That Can Vary
Creational	Abstract Factory	families of product objects
	Builder	how a composite object gets created
	Factory Method	subclass of object that is instantiated
	Prototype	class of object that is instantiated
	Singleton	the sole instance of a class
Structural	Adapter	interface to an object
	Bridge	implementation of an object
	Composite	structure and composition of an object
	Decorator	responsibilities of an object without subclassing
	Facade	interface to a subsystem
	Flyweight	storage costs of objects
	Proxy	how an object is accessed; its location
Behavioral	Chain of Responsibility	object that can fulfill a request
	Command	when and how a request is fulfilled
	Interpreter	grammar and interpretation of a language
	Iterator	how an aggregate's elements are accessed, traversed
	Mediator	how and which objects interact with each other
	Memento	what private information is stored outside an object, and when
	Observer	number of objects that depend on another object; how the dependent objects stay up to date
	State	states of an object
	Strategy	an algorithm
	Template Method	steps of an algorithm
	Visitor	operations that can be applied to object(s) without changing their class(es)

Creational patterns

- **Intro**: Creational design patterns abstract the instantiation process.They help make a system independent of how its objects are created,composed, and represented. A class creational pattern uses inheritance to vary the class that's instantiated, whereas an object creational pattern will delegate instantiation to another object.

  Creational patterns become important as systems evolve to depend more on object composition than class inheritance. As that happens,emphasis shifts away from hard-coding a fixed set of behaviors toward defining a smaller set of fundamental behaviors that can be composed into any number of more complex ones. Thus creating objects with particular behaviors requires more than simply instantiating a class.

  There are two recurring themes in these patterns. First, they all encapsulate knowledge about which concrete classes the system uses.Second, they hide how instances of these classes are created and put together. All the system at large knows about the objects is their interfaces as defined by abstract classes. Consequently, the creational patterns give you a lot of flexibility in what gets created, who creates it, how it gets created, and when. They let you configure a system with "product" objects that vary widely in structure and functionality. Configuration can be static (that is, specified at compile-time) or dynamic (at run-time).

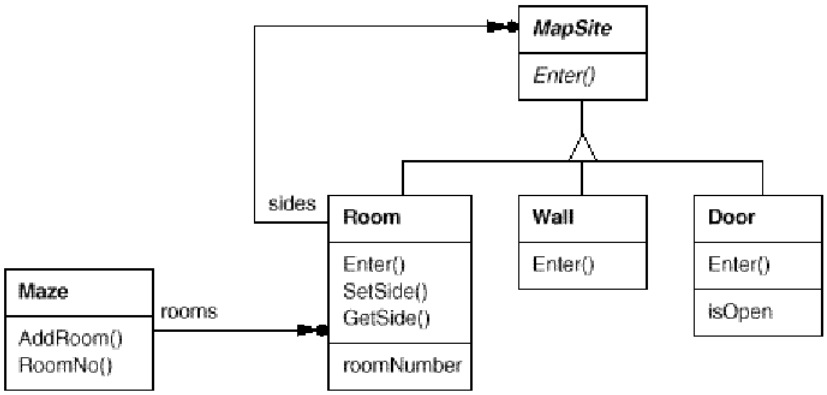
  Sometimes creational patterns are competitors. For example,there are cases when either Prototype (133)or Abstract Factory (99) could be used profitably. At other times they are complementary: Builder (110) can use one of the other patterns to implement which components get built. Prototype (133) can use Singleton (144) in its implementation.

  Because the creational patterns are closely related, we'll study all five of them together to highlight their similarities and differences.W

- **Maze game**: We'll use an example, building a maze for a computer game to illustrate their implementations. The maze and the game will vary slightly from pattern to pattern. Sometimes the game will be simply to find your way out of a maze; in that case the player will probably only have a local view of the maze. Sometimes mazes contain problems to solve and dangers to overcome, and these games may provide a map of the part of the maze that has been explored.

  We'll ignore many details of what can be in a maze and whether a maze game has a single or multiple players. Instead, we'll just focus on how mazes get created. We define a maze as a set of rooms. A room knows its neighbors; possible neighbors are another room, a wall, or a door to another room.

  The classes Room, Door, and Wall define the components of the maze used in all our examples. We define only the parts of these classes that are important for creating amaze. We'll ignore players, operations for displaying and wandering around in a maze, and other important functionality that isn't relevant to building the maze.



Each room has four sides. We use an enumeration Direction inC++ implementations to specify the north, south, east, and west sides of a room:

```
enum Direction {North, South, East, West};
```

The class MapSite is the common abstract class for all the components of a maze. To simplify the example, MapSite defines only one operation, Enter. Its meaning depends on what you're entering. If you enter a room, then your location changes. If you try to enter a door, then one of two things happen: If the door is open, you go into the next room. If the door is closed, then you hurt your nose.

```
class MapSite {
public:
    virtual void Enter() = 0;
};
```

Enter provides a simple basis for more sophisticated game operations. For example, if you are in a room and say "Go East," the game can simply determine which MapSite is immediately to the east and then call Enter on it. The subclass-specific Enter operation will figure out whether your location changed or your nose got hurt. In a real game, Enter could take the player object that's moving about as an argument.

Room is the concrete subclass of MapSite that defines the key relationships between components in the maze. It maintains references to other MapSite objects and stores a room number. The number will identify rooms in the maze.

```
class Room : public MapSite {
public:
    Room(int roomNo);

    MapSite* GetSide(Direction) const;
    void SetSide(Direction, MapSite*);

    virtual void Enter();

private:
    MapSite* _sides[4];
    int _roomNumber;
};
```

The following classes represent the wall or door that occurs on eachside of a room.

```
class Wall : public MapSite {
public:
    Wall();
    virtual void Enter();
}
```

```
class Door : public MapSite {
public:
    Door(Room* = 0, Room* = 0);
    virtual void Enter();
    Room* OtherSideFrom(Room*);
private:
    Room* _room1;
    Room* _room2;
    bool _isOpen;
};
```

We need to know about more than just the parts of a maze. We'll also define a Maze class to represent a collection of rooms.Maze can also find a particular room given a room number using its RoomNo operation.

```
class Maze {
public:
    Maze();
    void AddRoom(Room*);
    Room* RoomNo(int) const;
private:
    // ...
};
```

RoomNo could do a look-up using a linear search, a hash table,or even a simple array. But we won't worry about such details here.Instead, we'll focus on how to specify the components of a maze object.

Another class we define is MazeGame, which creates the maze.One straightforward way to create a maze is with a series of operations that add components to a maze and then interconnect them. For example, the following member function will create a maze consisting of two rooms with a door between them:

```
0  Maze* MazeGame::CreateMaze () {
1      Maze* aMaze = new Maze;
2
3      Room* r1 = new Room(1);
4      Room* r2 = new Room(2);
5
6      Door* theDoor = new Door(r1, r2);
7
8      aMaze->AddRoom(r1);
9      aMaze->AddRoom(r2);
10
11     r1->SetSide(North, new Wall);
12     r1->SetSide(East, theDoor);
13     r1->SetSide(South, new Wall);
14     r1->SetSide(West, new Wall);
15
16     r2->SetSide(North, new Wall);
17     r2->SetSide(East, new Wall);
18     r2->SetSide(South, new Wall);
19     r2->SetSide(West, theDoor);
20
21     return aMaze;
22 }
```

This function is pretty complicated, considering that all it does is create a maze with two rooms. There are obvious ways to make it simpler. For example, the Room constructor could initialize the sides with walls ahead of time. But that just

moves the code somewhere else.The real problem with this member function isn't its size but its inflexibility. It hard-codes the maze layout. Changing the layout means changing this member function, either by overriding it—which means reimplementing the whole thing—or by changing parts of it—which is error-prone and doesn't promote reuse.

The creational patterns show how to make this design more flexible, not necessarily smaller. In particular, they will make it easy to change the classes that define the components of a maze.

Suppose you wanted to reuse an existing maze layout for a new game containing (of all things) enchanted mazes. The enchanted maze game has new kinds of components, like **DoorNeedingSpell,** a door that can be locked and opened subsequently only with a spell; and **EnchantedRoom,** a room that can have unconventional items in it, like magic keys or spells. How can you change **CreateMaze** easily so that it creates mazes with these new classes of objects?

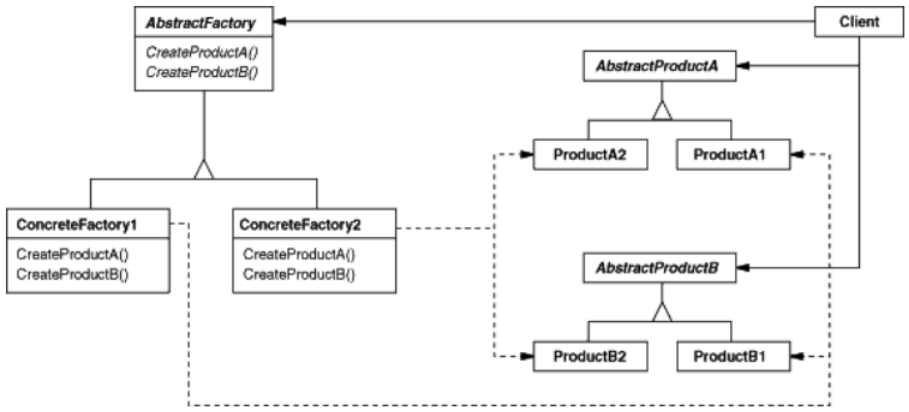
In this case, the biggest barrier to change lies in hard-coding the classes that get instantiated. The creational patterns provide different ways to remove explicit references to concrete classes from code that needs to instantiate them:

- If CreateMaze calls virtual functions instead of constructor calls to create the rooms, walls, and doors it requires, then you can change the classes that get instantiated by making a subclass of MazeGame and redefining those virtual functions. This approach is an example of the Factory Method (121) pattern.
- If CreateMaze is passed an object as a parameter to use to create rooms, walls, and doors, then you can change the classes of rooms, walls, and doors by passing a different parameter. This is an example of the Abstract Factory (99) pattern.
- If CreateMaze is passed an object that can create a new maze in its entirety using operations for adding rooms, doors, and walls to the maze it builds, then you can use inheritance to change parts of the maze or the way the maze is built. This is an example of the Builder (110) pattern.
- If CreateMaze is parameterized by various prototypical room,door, and wall objects, which it then copies and adds to the maze,then you can change the maze's composition by replacing these prototypical objects with different ones. This is an example of the Prototype (133) pattern.

The remaining creational pattern, Singleton (144), can ensure there's only one maze per game and that all game objects have ready access to it—without resorting to global variables or functions. Singleton also makes it easy to extend or replace the maze without touching existing code.

3.1 Abstract factory

- **Intent**: Provide an interface for creating families of related or dependent objects without specifying their concrete classes
- **AKA**: Kit
- **Motivation**: Consider a user interface toolkit that supports multiple look-and-feel standards, such as Motif and Presentation Manager. Different look-and-feels define different appearances and behaviors for user interface "widgets" like scroll bars, windows, and buttons. To be portable across look-and-feel standards, an application should not hard-code its widgets for a particular look and feel. Instantiating look-and-feel-specific classes of widgets throughout the application makes it hard to change the look and feel later.



- **Consequences**: The Abstract Factory pattern has the following benefits and liabilities:

  1. It isolates concrete classes. The Abstract Factory pattern helps you control the classes of objects that an application creates. Because a factory encapsulates the responsibility and the process of creating product objects, it isolates clients from implementation classes. Clients manipulate instances through their abstract interfaces. Product class names are isolated in the implementation of the concrete factory; they do not appear in client code.
  2. It makes exchanging product families easy. The class of a concrete factory appears only once in an application—that is, where it's instantiated. This makes it easy to change the concrete factory an application uses. It can use different product configurations simply by changing the concrete factory. Because an abstract factory creates a complete family of products, the whole product family changes at once. In our user interface example, we can switch from Motif widgets to Presentation Manager widgets simply by switching the corresponding factory objects and recreating the interface.
  3. It promotes consistency among products. When product objects in a family are designed to work together, it's important that an application use objects from only one family at a time. AbstractFactory makes this easy to enforce.

4. Supporting new kinds of products is difficult. Extending abstract factories to produce new kinds of Products isn't easy. That's because the AbstractFactory interface fixes the set of products that can be created. Supporting new kinds of products requires extending the factory interface, which involves changing the AbstractFactory class and all of its subclasses. We discuss one solution to this problem in the Implementation section.

- **Implementation**: Here are some useful techniques for implementing the Abstract Factory pattern.

    1. **Factories as singletons**: An application typically needs only one instance of a ConcreteFactory per product family. So it's usually best implemented as a Singleton (144).
    2. **Creating the products**: AbstractFactory only declares an interface for creating products. It's up to ConcreteProduct subclasses to actually create them. The most common way to do this is to define a factory method (see Factory Method (121)) for each product. A concrete factory will specify Its products by overriding the factory method for each. While this implementation is simple, it requires a new concrete factory subclass for each product family, even if the product families differ only slightly.

        If many product families are possible, the concrete factory can be implemented using the Prototype (133) pattern. The concrete factory is initialized with a prototypical instance of each product in the family, and it creates a new product by cloning its prototype. The Prototype-based approach eliminates the need for a new concrete factory class for each new product family
    3. **Defining extensible factories**: AbstractFactory usually defines a different operation for each kind of product it can produce. The kinds of products are encoded in the operation signatures. Adding a new kind of product requires changing the AbstractFactory interface and all the classes that depend on it.

        A more flexible but less safe design is to add a parameter to operations that create objects. This parameter specifies the kind of object to be created. It could be a class identifier, an integer, a string, or anything else that identifies the kind of product. In fact with this approach, AbstractFactory only needs a single "Make" operation with a parameter indicating the kind of object to create. This is the technique used in the Prototype- and the class-based abstract factories discussed earlier.

        But even when no coercion is needed, an inherent problem remains: All products are returned to the client with the same abstract interface as given by the return type. The client will not be able to differentiate or make safe assumptions about the class of a product. If clients need to perform subclass-specific operations, they won't be accessible through the abstract interface. Although the client could perform a downcast (e.g., with dynamic_cast in C++), that's not always feasible or safe, because the downcast can fail. This is the classic trade-off for a highly flexible and extensible interface.

- **Abstract Factory in the maze example**: Class MazeFactory can create components of mazes. It builds rooms, walls, and doors between rooms. It might be used by a program that reads plans for mazes from a file and builds the corresponding maze. Or it might be used by a program that builds mazes randomly. Programs that build mazes take a MazeFactory as an argument so that the programmer can specify the classes of rooms, walls, and doors to construct.

```
struct MazeFactory {
    MazeFactory();

    virtual Maze* MakeMaze() const
    { return new Maze; }
    virtual Wall* MakeWall() const
    { return new Wall; }
    virtual Room* MakeRoom(int n) const
    { return new Room(n); }
    virtual Door* MakeDoor(Room* r1, Room* r2) const
    { return new Door(r1, r2); }
};
```

Recall that the member function CreateMaze builds a small maze consisting of two rooms with a door between them. CreateMaze hard-codes the class names, making it difficult to create mazes with different components

Here's a version of CreateMaze that remedies that shortcoming by taking a MazeFactory as a parameter:

```
Maze* MazeGame::CreateMaze (MazeFactory& factory) {
    Maze* aMaze = factory.MakeMaze();
    Room* r1 = factory.MakeRoom(1);
    Room* r2 = factory.MakeRoom(2);

    Door* aDoor = factory.MakeDoor(r1, r2);

    aMaze->AddRoom(r1);
    aMaze->AddRoom(r2);

    r1->SetSide(North, factory.MakeWall());
    r1->SetSide(East, aDoor);
    r1->SetSide(South, factory.MakeWall());
    r1->SetSide(West, factory.MakeWall());

    r2->SetSide(North, factory.MakeWall());
    r2->SetSide(East, factory.MakeWall());
    r2->SetSide(South, factory.MakeWall());
    r2->SetSide(West, aDoor);

    return aMaze;
}
```

We can create EnchantedMazeFactory, a factory for enchanted mazes, by subclassing MazeFactory. EnchantedMazeFactory will override different member functions and return different subclasses of Room, Wall, etc.

```
class EnchantedMazeFactory : public MazeFactory {
    public:
    EnchantedMazeFactory();

    virtual Room* MakeRoom(int n) const
    { return new EnchantedRoom(n, CastSpell()); }

    virtual Door* MakeDoor(Room* r1, Room* r2) const
    { return new DoorNeedingSpell(r1, r2); }

    protected:
    Spell* CastSpell() const;
};
```

Now suppose we want to make a maze game in which a room can have a bomb set in it. If the bomb goes off, it will damage the walls (at least). We can make a subclass of Room keep track of whether the room has a bomb in it and whether the bomb has gone off. We'll also need a subclass of Wall to keep track of the damage done to the wall. We'll call these classes RoomWithABomb and BombedWall.

The last class we'll define is BombedMazeFactory, a subclass of MazeFactory that ensures walls are of class BombedWall and rooms are of class RoomWithABomb. BombedMazeFactory only needs to override two functions:

```
Wall* BombedMazeFactory::MakeWall () const {
    return new BombedWall;
}
Room* BombedMazeFactory::MakeRoom(int n) const {
    return new RoomWithABomb(n);
}
```

To build a simple maze that can contain bombs, we simply call CreateMaze with a BombedMazeFactory.

```
MazeGame game;
BombedMazeFactory factory;

game.CreateMaze(factory);
```

CreateMaze can take an instance of EnchantedMazeFactory just as well to build enchanted mazes.

Notice that the MazeFactory is just a collection of factory methods. This is the most common way to implement the Abstract Factory pattern. Also note that MazeFactory is not an abstract class; thus it acts as both the AbstractFactory and the ConcreteFactory. This is another common implementation for simple applications of the Abstract Factory pattern. Because the MazeFactory is a concrete class consisting entirely of factory methods, it's easy to make a new MazeFactory by making a subclass and overriding the operations that need to change.

CreateMaze used the SetSide operation on rooms to specify their sides. If it creates rooms with a BombedMazeFactory, then the maze will be made up of RoomWithABomb objects with BombedWall sides. If RoomWithABomb had to access a subclass-specific member of BombedWall, then it would have to cast a reference to its walls from Wall* to BombedWall*. This downcasting is safe as long as the argument is in fact a BombedWall, which is guaranteed to be true if walls are built solely with a BombedMazeFactory.

- **Known Uses**:
- **Related Patterns**: AbstractFactory classes are often implemented with factory methods (Factory Method (121)), but they can also be implemented using Prototype (133).

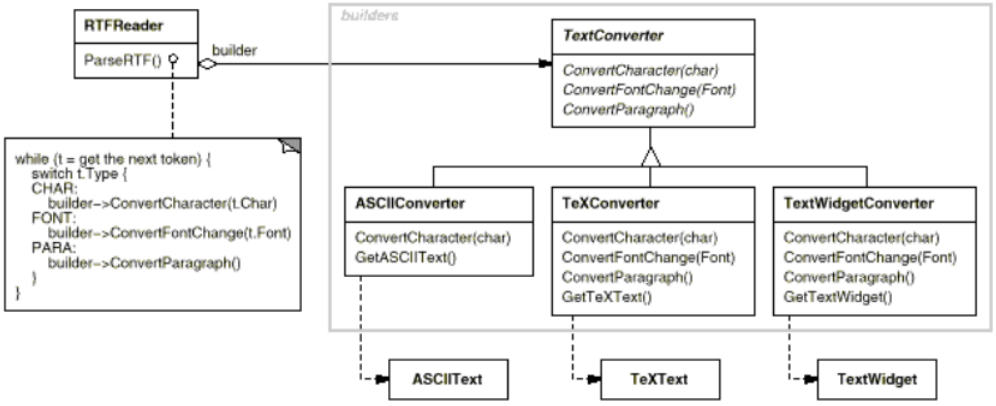
  A concrete factory is often a singleton (Singleton (144)).

3.2 Builder

- **Intent**: Separate the construction of a complex object from its representation so that the same construction process can create different representations
- **Motivation**: A reader for the RTF (Rich Text Format) document exchange format should be able to convert RTF to many text formats. The reader might convert RTF documents into plain ASCII text or into a text widget that can be edited interactively. The problem, however, is that the number of possible conversions is open-ended. So it should be easy to add a new conversion without modifying the reader.

  A solution is to configure the RTFReader class with a TextConverter object that converts RTF to another textual representation. As the RTFReader parses the RTF document, it uses the TextConverter to perform the conversion. Whenever the RTF-Reader recognizes an RTF token (either plain text or an RTF control word), it issues a request to the TextConverter to convert the token. TextConverter objects are responsible both for performing the data conversion and for representing the token in a particular format

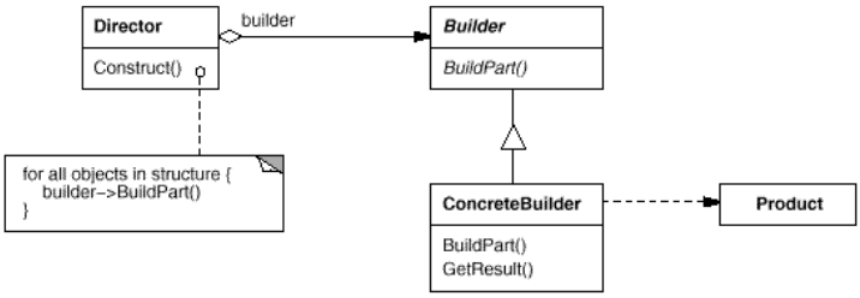
  Subclasses of TextConverter specialize in different conversions and formats. For example, an ASCIIConverter ignores requests to convert anything except plain text. A TeXConverter, on the other hand, will implement operations for all requests in order to produce a TeX representation that captures all the stylistic information in the text. A TextWidgetConverter will produce a complex user interface object that lets the user see and edit the text.



Each kind of converter class takes the mechanism for creating and assembling a complex object and puts it behind an abstract interface. The converter is separate from the reader, which is responsible for parsing an RTF document.

The Builder pattern captures all these relationships. Each converter class is called a builder in the pattern, and the reader is called the director. Applied to this example, the Builder pattern separates the algorithm for interpreting a textual format (that is, the parser for RTF documents) from how a converted format gets created and represented. This lets us reuse the RTFReader's parsing algorithm to create different text representations from RTF documents—just configure the RTFReader with different subclasses of TextConverter.

- **Applicability**: Use the builder pattern when
  - the algorithm for creating a complex object should be independent of the parts that make up the object and how they're assembled.
  - the construction process must allow different representations for the object that's constructed.
- **Structure**:

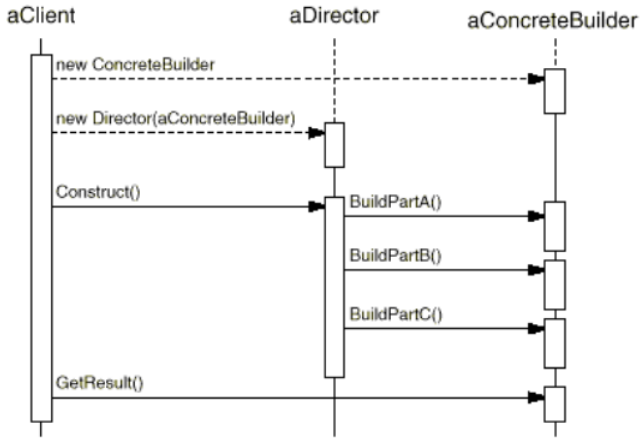


- **Participants**:
  - **Builder (TextConverter)**: specifies an abstract interface for creating parts of a Product object.
  - **ConcreteBuilder (ASCIIConverter, TeXConverter, TextWidgetConverter)**: constructs and assembles parts of the product by implementing the Builder interface.

    defines and keeps track of the representation it creates.

    provides an interface for retrieving the product (e.g., GetASCIIText, GetTextWidget).
  - **Director (RTFReader)**: constructs an object using the Builder interface.
  - **Product (ASCIIText, TeXText, TextWidget)**: represents the complex object under construction. ConcreteBuilder builds the product's internal representation and defines the process by which it's assembled.

    includes classes that define the constituent parts, including interfaces for assembling the parts into the final result.
- **Collaborations**:
  - The client creates the Director object and configures it with the desired Builder object.
  - Director notifies the builder whenever a part of the product should be built.
  - Builder handles requests from the director and adds parts to the product.
  - The client retrieves the product from the builder



- **Consequences**: Here are key consequences of the Builder pattern:

  1. It lets you vary a product's internal representation. The Builder object provides the director with an abstract interface for constructing the product. The interface lets the builder hide the representation and internal structure of the product. It also hides how the product gets assembled. Because the product is constructed through an abstract interface, all you have to do to change the product's internal representation is define a new kind of builder.
  2. It isolates code for construction and representation. The Builder pattern improves modularity by encapsulating the way a complex object is constructed and represented. Clients needn't know anything about the classes that define the product's internal structure; such classes don't appear in Builder's interface.

     Each ConcreteBuilder contains all the code to create and assemble a particular kind of product. The code is written once; then different Directors can reuse it to build Product variants from the same set of parts. In the earlier RTF example, we could define a reader for a format other than RTF, say, an SGMLReader, and use the same TextConverters to generate ASCIIText, TeXText, and TextWidget renditions of SGML documents.
  3. It gives you finer control over the construction process. Unlike creational patterns that construct products in one shot, the Builder pattern constructs the product step by step under the director's control. Only when the product is finished does the director retrieve it from the builder. Hence the Builder interface reflects the process of constructing the product more than other creational patterns. This gives you finer control over the construction process and consequently the internal structure of the resulting product.

- **Implementation**: Typically there's an abstract Builder class that defines an operation for each component that a director may ask it to create. The operations do nothing by default. A ConcreteBuilder class overrides operations for components it's interested in creating.

  Here are other implementation issues to consider:

  1. Assembly and construction interface. Builders construct their products in step-by-step fashion. Therefore the Builder class interface must be general enough to allow the construction of products for all kinds of concrete builders.

A key design issue concerns the model for the construction and assembly process. A model where the results of construction requests are simply appended to the product is usually sufficient. In the RTF example, the builder converts and appends the next token to the text it has converted so far.

But sometimes you might need access to parts of the product constructed earlier. In the Maze example we present in the Sample Code, the MazeBuilder interface lets you add a door between existing rooms. Tree structures such as parse trees that are built bottom-up are another example. In that case, the builder would return child nodes to the director, which then would pass them back to the builder to build the parent nodes.

2. Why no abstract class for products? In the common case, the products produced by the concrete builders differ so greatly in their representation that there is little to gain from giving different products a common parent class. In the RTF example, the ASCIIText and the TextWidget objects are unlikely to have a common interface, nor do they need one. Because the client usually configures the director with the proper concrete builder, the client is in a position to know which concrete subclass of Builder is in use and can handle its products accordingly.
3. Empty methods as default in Builder. In C++, the build methods are intentionally not declared pure virtual member functions. They're defined as empty methods instead, letting clients override only the operations they're interested in. A concrete subclass must implement all inherited pure virtual methods.

   If the base class methods are virtual but not pure virtual, then they already have implementations. A derived class may override only the ones it wants, and it can still be instantiated.

- **Sample Code**: We'll define a variant of the CreateMaze member function that takes a builder of class MazeBuilder as an argument

  The MazeBuilder class defines the following interface for building mazes:

```
0  class MazeBuilder {
1  public:
2      virtual void BuildMaze() { }
3      virtual void BuildRoom(int room) { }
4      virtual void BuildDoor(int roomFrom, int roomTo) { }
5
6      virtual Maze* GetMaze() { return 0; }
7
8  protected:
9      MazeBuilder();
10 };
```

  This interface can create three things: (1) the maze, (2) rooms with a particular room number, and (3) doors between numbered rooms. The GetMaze operation returns the maze to the client. Subclasses of MazeBuilder will override this operation to return the maze that they build.

  All the maze-building operations of MazeBuilder do nothing by default. They're not declared pure virtual to let derived classes override only those methods in which they're interested.

Given the MazeBuilder interface, we can change the CreateMaze member function to take this builder as a parameter.

Given the MazeBuilder interface, we can change the CreateMaze member function to take this builder as a parameter.

```
Maze* MazeGame::CreateMaze (MazeBuilder& builder) {
    builder.BuildMaze();

    builder.BuildRoom(1);
    builder.BuildRoom(2);
    builder.BuildDoor(1, 2);

    return builder.GetMaze();
}
```

Compare this version of CreateMaze with the original. Notice how the builder hides the internal representation of the Maze—that is, the classes that define rooms, doors, and walls—and how these parts are assembled to complete the final maze. Someone might guess that there are classes for representing rooms and doors, but there is no hint of one for walls. This makes it easier to change the way a maze is represented, since none of the clients of MazeBuilder has to be changed.

Like the other creational patterns, the Builder pattern encapsulates how objects get created, in this case through the interface defined by MazeBuilder. That means we can reuse MazeBuilder to build different kinds of mazes. The CreateComplexMaze operation gives an example:

Note that MazeBuilder does not create mazes itself; its main purpose is just to define an interface for creating mazes. It defines empty implementations primarily for convenience. Subclasses of MazeBuilder do the actual work.

The subclass StandardMazeBuilder is an implementation that builds simple mazes. It keeps track of the maze it's building in the variable _currentMaze.

```
class StandardMazeBuilder : public MazeBuilder {
public:
    StandardMazeBuilder();

    virtual void BuildMaze();
    virtual void BuildRoom(int);
    virtual void BuildDoor(int, int);

    virtual Maze* GetMaze();

private:
    Direction CommonWall(Room*, Room*);
    Maze* _currentMaze;
};
```

CommonWall is a utility operation that determines the direction of the common wall between two rooms.

The StandardMazeBuilder constructor simply initializes _currentMaze.

```
StandardMazeBuilder::StandardMazeBuilder () {
    _currentMaze = 0;
}
```

BuildMaze instantiates a Maze that other operations will assemble and eventually return to the client (with GetMaze).

```
void StandardMazeBuilder::BuildMaze () {
    _currentMaze = new Maze;
}

Maze* StandardMazeBuilder::GetMaze () {
    return _currentMaze;
}
```

The BuildRoom operation creates a room and builds the walls around it:

```
void StandardMazeBuilder::BuildRoom (int n) {
    if (!_currentMaze->RoomNo(n)) {
        Room* room = new Room(n);
        _currentMaze->AddRoom(room);

        room->SetSide(North, new Wall);
        room->SetSide(South, new Wall);
        room->SetSide(East, new Wall);
        room->SetSide(West, new Wall);
    }
}
```

To build a door between two rooms, StandardMazeBuilder looks up both rooms in the maze and finds their adjoining wall:

```
void StandardMazeBuilder::BuildDoor (int n1, int n2) {
    Room* r1 = _currentMaze->RoomNo(n1);
    Room* r2 = _currentMaze->RoomNo(n2);
    Door* d = new Door(r1, r2);
    r1->SetSide(CommonWall(r1,r2), d);
    r2->SetSide(CommonWall(r2,r1), d);
}
```

Clients can now use CreateMaze in conjunction with StandardMazeBuilder to create a maze:

```
Maze* maze;
MazeGame game;
StandardMazeBuilder builder;

game.CreateMaze(builder);
maze = builder.GetMaze();
```

We could have put all the StandardMazeBuilder operations in Maze and let each Maze build itself. But making Maze smaller makes it easier to understand and modify, and StandardMazeBuilder is easy to separate from Maze. Most importantly, separating the two lets you have a variety of MazeBuilders, each using different classes for rooms, walls, and doors.

A more exotic MazeBuilder is CountingMazeBuilder. This builder doesn't create a maze at all; it just counts the different kinds of components that would have been created.

```
class CountingMazeBuilder : public MazeBuilder {
public:
    CountingMazeBuilder();

    virtual void BuildMaze();
    virtual void BuildRoom(int);
    virtual void BuildDoor(int, int);
    virtual void AddWall(int, Direction);

    void GetCounts(int&, int&) const;
private:
    int _doors;
    int _rooms;
};
```

The constructor initializes the counters, and the overridden MazeBuilder operations increment them accordingly.

```
CountingMazeBuilder::CountingMazeBuilder () {
    _rooms = _doors = 0;
}

void CountingMazeBuilder::BuildRoom (int) {
    _rooms++;
}

void CountingMazeBuilder::BuildDoor (int, int) {
    _doors++;
}

void CountingMazeBuilder::GetCounts (int& rooms, int& doors)
    const {
    rooms = _rooms;
    doors = _doors;
}
```

Here's how a client might use a CountingMazeBuilder:

```
int rooms, doors;
MazeGame game;
CountingMazeBuilder builder;

game.CreateMaze(builder);
builder.GetCounts(rooms, doors);

cout << "The maze has "
    << rooms << " rooms and "
    << doors << " doors" << '\n';
```

- **Related patterns**: Abstract Factory (99) is similar to Builder in that it too may construct complex objects. The primary difference is that the Builder pattern focuses on constructing a complex object step by step. Abstract Factory's emphasis is on families of product objects (either simple or complex). Builder returns the product as a final step, but as far as the Abstract Factory pattern is concerned, the product gets returned immediately.

  A Composite (183) is what the builder often builds.

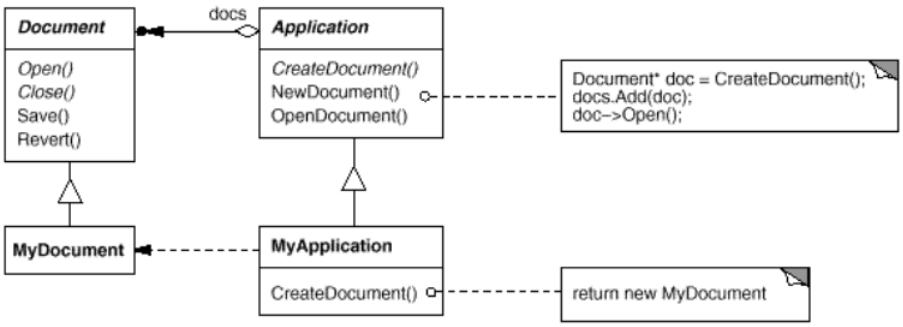
3.3 Factory Method

- **Intent**: Define an interface for creating an object, but let subclasses decide which class to instantiate. Factory Method lets a class defer instantiation to subclasses.
- **Also known as**: Virtual constructor
- **Motivation**: Frameworks use abstract classes to define and maintain relationships between objects. A framework is often responsible for creating these objects as well.

  Consider a framework for applications that can present multiple documents to the user. Two key abstractions in this framework are the classes Application and Document. Both classes are abstract, and clients have to subclass them to realize their application-specific implementations. To create a drawing application, for example, we define the classes DrawingApplication and DrawingDocument. The Application class is responsible for managing Documents and will create them as required—when the user selects Open or New from a menu, for example.

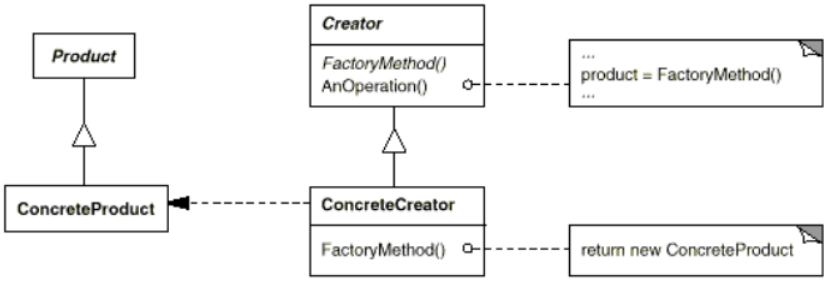
  Because the particular Document subclass to instantiate is application-specific, the Application class can't predict the subclass of Document to instantiate—the Application class only knows when a new document should be created, not what kind of Document to create. This creates a dilemma: The framework must instantiate classes, but it only knows about abstract classes, which it cannot instantiate.

  The Factory Method pattern offers a solution. It encapsulates the knowledge of which Document subclass to create and moves this knowledge out of the framework.



  Application subclasses redefine an abstract CreateDocument operation on Application to return the appropriate Document subclass. Once an Application subclass is instantiated, it can then instantiate application-specific Documents without knowing their class. We call CreateDocument a **factory method** because it's responsible for "manufacturing" an object

- **Applicability**: Use the Factory Method pattern when
  - a class can't anticipate the class of objects it must create.
  - a class wants its subclasses to specify the objects it creates.
  - classes delegate responsibility to one of several helper subclasses, and you want to localize the knowledge of which helper subclass is the delegate.
- **Structure**:



- **Participants**:
  - **Product (Document)**: defines the interface of objects the factory method creates.
  - **ConcreteProduct (MyDocument)**: implements the Product interface.
  - **Creator (Application)**: declares the factory method, which returns an object of type Product. Creator may also define a default implementation of the factory method that returns a default ConcreteProduct object.

    may call the factory method to create a Product object.
  - **ConcreteCreator (MyApplication)**: overrides the factory method to return an instance of a ConcreteProduct
- **Collaborations**: Creator relies on its subclasses to define the factory method so that it returns an instance of the appropriate ConcreteProduct.
- **Consequences**: Factory methods eliminate the need to bind application-specific classes into your code. The code only deals with the Product interface; therefore it can work with any user-defined ConcreteProduct classes.

  A potential disadvantage of factory methods is that clients might have to subclass the Creator class just to create a particular ConcreteProduct object. Subclassing is fine when the client has to subclass the Creator class anyway, but otherwise the client now must deal with another point of evolution.

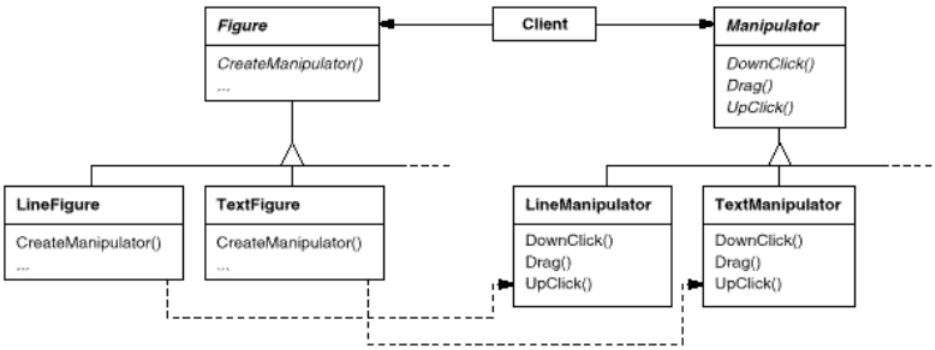
  Here are two additional consequences of the Factory Method pattern:

  1. **Provides hooks for subclasses**: Creating objects inside a class with a factory method is always more flexible than creating an object directly. Factory Method gives subclasses a hook for providing an extended version of an object.

     In the Document example, the Document class could define a factory method called CreateFileDialog that creates a default file dialog object for opening an existing document. A Document subclass can define an application-specific file dialog by overriding this factory method. In this case the factory method is not abstract but provides a reasonable default implementation.
  2. **Connects parallel class hierarchies**: In the examples we've considered so far, the factory method is only called by Creators. But this doesn't have to be the case; clients can find factory methods useful, especially in the case of parallel class hierarchies.

Parallel class hierarchies result when a class delegates some of its responsibilities to a separate class. Consider graphical figures that can be manipulated interactively; that is, they can be stretched, moved, or rotated using the mouse. Implementing such interactions isn't always easy. It often requires storing and updating information that records the state of the manipulation at a given time. This state is needed only during manipulation; therefore it needn't be kept in the figure object. Moreover, different figures behave differently when the user manipulates them. For example, stretching a line figure might have the effect of moving an endpoint, whereas stretching a text figure may change its line spacing.

With these constraints, it's better to use a separate Manipulator object that implements the interaction and keeps track of any manipulation-specific state that's needed. Different figures will use different Manipulator subclasses to handle particular interactions. The resulting Manipulator class hierarchy parallels (at least partially) the Figure class hierarchy:



The Figure class provides a CreateManipulator factory method that lets clients create a Figure's corresponding Manipulator. Figure subclasses override this method to return an instance of the Manipulator subclass that's right for them. Alternatively, the Figure class may implement CreateManipulator to return a default Manipulator instance, and Figure subclasses may simply inherit that default. The Figure classes that do so need no corresponding Manipulator subclass—hence the hierarchies are only partially parallel

Notice how the factory method defines the connection between the two class hierarchies. It localizes knowledge of which classes belong together.

- **Implementation**: Consider the following issues when applying the Factory Method pattern:

  1. **Two major varieties**: The two main variations of the Factory Method pattern are (1) the case when the Creator class is an abstract class and does not provide an implementation for the factory method it declares, and (2) the case when the Creator is a concrete class and provides a default implementation for the factory method. It's also possible to have an abstract class that defines a default implementation, but this is less common.

The first case requires subclasses to define an implementation, because there's no reasonable default. It gets around the dilemma of having to instantiate unforeseeable classes. In the second case, the concrete Creator uses the factory method primarily for flexibility. It's following a rule that says, "Create objects in a separate operation so that subclasses can override the way they're created." This rule ensures that designers of subclasses can change the class of objects their parent class instantiates if necessary.

2. **Parameterized factory methods**: Another variation on the pattern lets the factory method create multiple kinds of products. The factory method takes a parameter that identifies the kind of object to create. All objects the factory method creates will share the Product interface. In the Document example, Application might support different kinds of Documents. You pass CreateDocument an extra parameter to specify the kind of document to create.

   The Unidraw graphical editing framework uses this approach for reconstructing objects saved on disk. Unidraw defines a Creator class with a factory method Create that takes a class identifier as an argument. The class identifier specifies the class to instantiate. When Unidraw saves an object to disk, it writes out the class identifier first and then its instance variables. When it reconstructs the object from disk, it reads the class identifier first.

   Once the class identifier is read, the framework calls Create, passing the identifier as the parameter. Create looks up the constructor for the corresponding class and uses it to instantiate the object. Last, Create calls the object's Read operation, which reads the remaining information on the disk and initializes the object's instance variables. A parameterized factory method has the following general form, where

   MyProduct and YourProduct are subclasses of Product:

```
class Creator {
    public:
    virtual Product* Create(ProductId);
};

Product* Creator::Create (ProductId id) {
    if (id == MINE) return new MyProduct;
    if (id == YOURS) return new YourProduct;
    // repeat for remaining products...
    return 0;
}
```

   Overriding a parameterized factory method lets you easily and selectively extend or change the products that a Creator produces. You can introduce new identifiers for new kinds of products, or you can associate existing identifiers with different products.

   For example, a subclass MyCreator could swap MyProduct and YourProduct and support a new TheirProduct subclass:

```
Product* MyCreator::Create (ProductId id) {
    if (id == YOURS) return new MyProduct;
    if (id == MINE) return new YourProduct;
    // N.B.: switched YOURS and MINE

    if (id == THEIRS) return new TheirProduct;

    return Creator::Create(id); // called if all others
    ↪  fail
}
```

Notice that the last thing this operation does is call Create on the parent class. That's because MyCreator::Create handles only YOURS, MINE, and THEIRS differently than the parent class. It isn't interested in other classes. Hence MyCreator extends the kinds of products created, and it defers responsibility for creating all but a few products to its parent.

3. **Language-specific variants and issues**: Different languages lend themselves to other interesting variations and caveats.
4. **Using templates to avoid subclassing**: As we've mentioned, another potential problem with factory methods is that they might force you to subclass just to create the appropriate Product objects. Another way to get around this in C++ is to provide a template subclass of Creator that's parameterized by the Product class:

```
class Creator {
    public:
    virtual Product* CreateProduct() = 0;
};

template <class TheProduct>
class StandardCreator: public Creator {
    public:
    virtual Product* CreateProduct();
};

template <class TheProduct>
Product* StandardCreator<TheProduct>::CreateProduct () {
    return new TheProduct;
}
```

With this template, the client supplies just the product class—no subclassing of Creator is required.

```
class MyProduct : public Product {
    public:
    MyProduct();
    // ...
};

StandardCreator<MyProduct> myCreator;
```

5. **Naming conventions**. It's good practice to use naming conventions that make it clear you're using factory methods. For example, the MacApp Macintosh application framework always declares the abstract operation that defines the factory method as Class* DoMakeClass(), where Class is the Product class.

- **Sample Code**: The function CreateMaze builds and returns a maze. One problem with this function is that it hard-codes the classes of maze, rooms, doors, and walls. We'll introduce factory methods to let subclasses choose these components

  First we'll define factory methods in MazeGame for creating the maze, room, wall, and door objects:

```
class MazeGame {
public:
    Maze* CreateMaze();

    // factory methods:

    virtual Maze* MakeMaze() const
        { return new Maze; }
    virtual Room* MakeRoom(int n) const
        { return new Room(n); }
    virtual Wall* MakeWall() const
        { return new Wall; }
    virtual Door* MakeDoor(Room* r1, Room* r2) const
        { return new Door(r1, r2); }
};
```

  Each factory method returns a maze component of a given type. MazeGame provides default implementations that return the simplest kinds of maze, rooms, walls, and doors.

  Now we can rewrite CreateMaze to use these factory methods:

```
Maze* MazeGame::CreateMaze () {
    Maze* aMaze = MakeMaze();

    Room* r1 = MakeRoom(1);
    Room* r2 = MakeRoom(2);
    Door* theDoor = MakeDoor(r1, r2);

    aMaze->AddRoom(r1);
    aMaze->AddRoom(r2);

    r1->SetSide(North, MakeWall());
    r1->SetSide(East, theDoor);
    r1->SetSide(South, MakeWall());
    r1->SetSide(West, MakeWall());

    r2->SetSide(North, MakeWall());
    r2->SetSide(East, MakeWall());
    r2->SetSide(South, MakeWall());
    r2->SetSide(West, theDoor);

    return aMaze;
}
```

Different games can subclass MazeGame to specialize parts of the maze. MazeGame subclasses can redefine some or all of the factory methods to specify variations in products. For example, a BombedMazeGame can redefine the Room and Wall products to return the bombed varieties:

3.4 Prototype